Week 3 - Stream and Block Ciphers

MAT260: Cryptology

Gary Hobson

Date May 19, 2025

Introduction:

The assigned computer problems this week are from Chapter 5, Problems 1, 3, and Chapter 6, Problem 1.

Make sure to review the example computer problems in "Appendix C.4 Examples for Chapter 5" and "Appendix C.5 Examples for Chapter 6" that work similar problems to those you are assigned.

Make sure to run your code so all relevant computations/results are displayed and then export your work as a PDF file for submission.

Chapter 5 Problems:

Problem 1:

To determine the recurrence that generated the given binary sequence, I first input the full sequenec into MATLAB. I used the lfsrlength function to estimate the minimal order of the linear feedback shift register (LFSR), which turned out to be 6. Then, I Applied the lfsrsolve function to solve for the feedback coefficients that define the recurrence reltaion. The solution indicated that the relation is:

xn+6 = xn + xn+1 + xn+3 + xn+4 mod 2

```
% Define the sequence
seq = [1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, ...
        0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, ...
        0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1];

%Guess the length
lfsrlength(seq, 20)
```

```
Order   Determinant
1        1
2        1
```

```
3        1
4        0
5        1
6        1
7        0
8        0
9        0
10        0
11        0
12        0
13        0
14        0
15        2
16        4.5475e-13
17        9.0949e-13
18        5.457e-12
19        2.9104e-11
20        7.276e-12
```

```
coeffs = lfsrsolve(seq, 6)
```

```
coeffs = 1×6
     1     1     0     1     1     0
```

```
%The recurrence relation is:
fprintf('x(n+6) = ');
```

```
x(n+6) =
```

```
for i = 1:length(coeffs)
    if coeffs(i)
        fprintf(' + x(n+%d)', i-1);
    end
end
```

```
 + x(n+0) + x(n+1) + x(n+3) + x(n+4)
```

```
fprintf(' mod 2\n');
```

```
 mod 2
```

Problem 3:

To solve the problem, I used a plaintext atack on an XOR based ciphertext encrypted with a linear feedback shift register (LFSR). First, I XORed the known segment of the plaintext with the coresponding ciphertext bits to recover the initial portion of the LFSR keystream. I thenused the lfsrlength function to estimate the LFSR order and confirmed that a 5 bit LFSR was likely. Using lfsrsolve, I determined the recurrence relation coefficients. Finally, I regenerated the full keystream with these coefficients and decrypted the full ciphertext by XORing it with the keystream, I believe successfully

recovering the original plaintext.

```
% Known portion of the plaintext and ciphertext
plaintext_known = [1 0 0 1 0 0 1 0 0 1 0 0 1 0 0];
ciphertext = [0 1 1 0 0 0 1 0 1 0 1 1 1 0 0 1 1 1 0, ...
              1 0 1 0 0 0 1 0 0 0 1 1 0 0 0 1 0 1 0, ...
              1 1 1 0 0 1 1 1 0 1 0 1];

% Get LFSR keystream start
lfsr_start = mod(plaintext_known + ciphertext(1:length(plaintext_known)), 2);

% Guess LFSR length
lfsrlength(lfsr_start, 8);  % Visual inspection suggests length is 5
```

```
Order   Determinant
1        1
2        0
3        0
4        1
5        1
6        0
7        0
8        0
```

```
% Solve for recurrence coefficients
coeffs = lfsrsolve(lfsr_start, 5);

% Generate full LFSR sequence (same length as ciphertext)
keystream = lfsr(coeffs, lfsr_start(1:5), length(ciphertext));

% Recover full plaintext
plaintext_full = mod(ciphertext + keystream, 2);

% Recovered Plaintext
disp(plaintext_full);
```

```
     1     0     0     1     0     0     1     0     0     1     0     0     1     0     0     1     0     0     1
```

Chapter 6 Problems:

Problem 1:

I tackled the Hill cipher decryption problem by writing a MATLAB script that takes the ciphertext "zirkzwopjjoptfapuhfhadrq" and turns it into the

plaintext "jackandjillwentupthehill" using the given 4x4 matrix. I started by converting the ciphertext into numbers and organizing them into blocks,

then figured out the matrix inverse modulo 26 with a custom adjugate function and symbolic math to keep things precise, tweaking the code

along the way to fix issues like the missing adjugate function and making sure the blocks and matrix math lined up right.

```
% Define the encryption matrix M
M = [1 2 3 4; 4 3 2 1; 11 2 4 6; 2 9 6 4];

% Convert ciphertext to numbers (a=0, b=1, ..., z=25)
ciphertext = 'zirkzwopjjoptfapuhfhadrq';
nums = double(ciphertext) - double('a'); % 1x24 vector
nums = reshape(nums, 4, 6)'; % 6x4 matrix, each row is a block of 4 letters

% Function to compute adjugate matrix
function adjM = compute_adjugate(A)
    n = size(A, 1);
    adjM = sym(zeros(n, n)); % Use symbolic for precision
    for i = 1:n
        for j = 1:n
            % Compute minor by removing row i and column j
            subM = A;
            subM(i,:) = [];
            subM(:,j) = [];
            % Cofactor: (-1)^(i+j) * det(subM)
            cof = (-1)^(i+j) * det(subM);
            adjM(j,i) = cof; % Place in transposed position
        end
    end
end

% Function to compute modular inverse
function inv = mod_inverse(a, m)
    % Find x such that a * x ≡ 1 (mod m) using extended Euclidean algorithm
    [g, x, ~] = gcd(a, m);
    if g ~= 1
        error('Modular inverse does not exist');
    end
    inv = mod(x, m); % Ensure positive
    if inv < 0
        inv = inv + m;
    end
end

% Compute inverse matrix M_inv modulo 26
M_sym = sym(M); % Symbolic matrix for exact arithmetic
det_M = det(M_sym); % Should be -55
det_M_mod = mod(det_M, 26); % Should be 23
inv_det_M = mod_inverse(det_M_mod, 26); % Should be 17
adjM_sym = compute_adjugate(M_sym); % Adjugate matrix
M_inv = double(mod(adjM_sym * inv_det_M, 26)); % M^{-1} mod 26

% Decrypt: p = c * M_inv mod 26
```

```
plaintext_nums = mod(nums * M_inv, 26); % 6x4 matrix
plaintext_nums = reshape(plaintext_nums', 1, 24); % Flatten to 1x24, transpose to
match order
plaintext = char(plaintext_nums + double('a')); % Convert to letters
disp('Decrypted plaintext:');
```

```
Decrypted plaintext:
```

```
disp(plaintext);
```

```
jackandjillwentupthehill
```